

Simulated Annealing as a solver of Constraint Satisfaction Problems

1. Integration of KSAT and Simulated Annealing

The core of this project is the collaboration between KSAT and Simulated Annealing, whereby the former provides the problem definition and the latter the optimization framework. Next, we will describe how these components interact, in particular how the structure of KSAT has been built in order to guarantee always a correct interaction with SA.

Core Functionalities of KSAT

The interaction between KSAT and Simulated Annealing is facilitated by the definition of the following critical methods, which support the Simulated Annealing process:

The **cost()** function calculates the total number of unsatisfied clauses in the current configuration. It utilizes the provided formula, in which the satisfiability of each sentence is established by assessing the literals' signs (s) and current variable assignments (x). If a clause is unsatisfied, its contribution to the cost is one; otherwise, it is zero. The function adds these contributions to get the overall cost, which shows how distant the configuration is from meeting all clauses. When certain sentences are affected (e.g., during a move), the function recalculates the cost solely for those phrases to maximize efficiency.

The **compute_delta_cost()** function determines the change in cost (Δ Cost) if a specific move is made, where a move flips the value of a single variable in the configuration. It identifies the clauses impacted by the flipped variable and calculates their contributions to the cost before and after the flip. The difference between these contributions represents the delta cost, which is used to decide whether the proposed move should be accepted during the optimization process. The use of *np.prod* in a vectorized manner allows efficient computation of the product of terms across clauses, enabling the simultaneous evaluation of satisfiability for multiple clauses without explicit loops.

The **propose_move()** function selects a random variable to flip in the current configuration. Through randomness we make sure that the optimization explores a various range of configurations, avoiding local minima.

The **accept_move()** function applies a proposed move by flipping the sign of the selected variable in the configuration. This modifies the current configuration: either improving the cost (if the move is beneficial) or potentially increasing it (if the move is accepted based on the SA probabilistic acceptance rule).

The modular structure enhances scalability, enabling independent testing and refinement of key components, such as the *cost()* and *compute_delta_cost()* methods, without impacting the overall framework. Furthermore, the integration of *compute_delta_cost()* significantly improves optimization efficiency by focusing only on affected clauses during computation, thereby reducing overhead and enabling the system to handle larger problem sizes effectively.

2. Analysis of Simulated Annealing for the 3-SAT problem

Trial-and-Error Approach

We used a trial-and-error approach, which is a foundational method in exploring and fine-tuning parameters for optimization algorithms, especially in complex problems like 3-SAT. This approach involves iteratively testing various parameter configurations, observing the results, and refining choices based on empirical performance.

Parameters Choice

The chosen parameters for the Simulated Annealing algorithm were:

- `mcmc_steps` = 1000
- `anneal_steps` = 85
- $\beta_0 = 1.0$, $\beta_1 = 10.0$

They provide a consistent approach to solving the 3-SAT problem by balancing exploration and exploitation during optimization. The `anneal_steps` parameter, set to 85, introduces a finer granularity in the cooling schedule, allowing a smoother and more controlled transition from high randomness (exploration) to focused search (exploitation). This ensures a more thorough exploration of the solution space and improves the likelihood of escaping local minima. The `mcmc_steps` = 1000 ensures that at each temperature, a sufficient number of Monte Carlo moves are performed to adequately sample the configuration space.

Evolution in Acceptance Rate

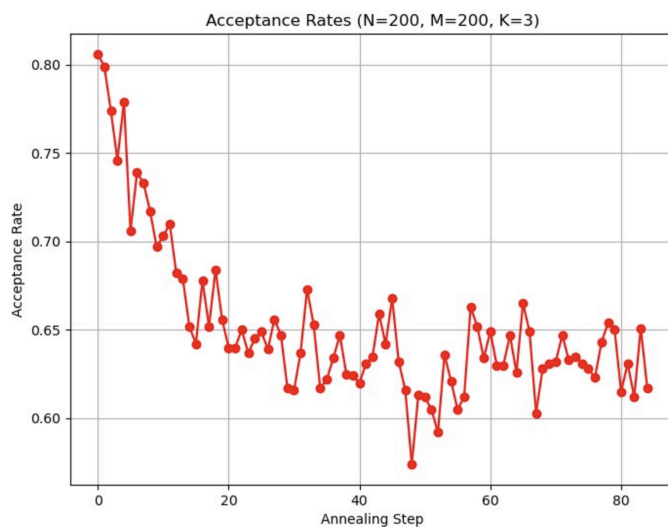


Figure 1: Evolution of Acceptance Rate with $N, M = 200, K = 3$.

Figure 1 illustrates the evolution of the acceptance rate during the annealing schedule for solving a 3-SAT problem with $N, M = 200$. The acceptance rate represents the proportion of proposed moves that are accepted at each annealing step. The simulated annealing algorithm was executed with the parameters mentioned in the above paragraph (Parameters Choice).

Initially, the acceptance rate is high (~80%), reflecting the algorithm's greater tolerance for moves that increase the cost due to the low initial value of β ($\beta_0=1.0$). As the annealing progresses, β increases linearly toward its final value ($\beta_1=10.0$), reducing the probability of accepting cost-increasing moves. Consequently, the acceptance rate gradually declines, stabilizing around 60% in the later stages.

This behavior aligns with the theoretical foundation of the simulated annealing algorithm, which mimics the physical process of annealing in materials. At higher "temperatures" (lower β), the system explores the solution space more freely, allowing it to escape local minima. As the "temperature" decreases (higher β), the algorithm focuses on refining the current solution by accepting only advantageous or minor cost-increasing moves with lower probability. This trade-off between exploration and exploitation enables simulated annealing to balance global and local search strategies effectively.

The observed fluctuations in the acceptance rate reflect the stochastic nature of the algorithm, where random moves and cost changes influence the acceptance probability. Overall, the graph demonstrates the expected behavior of simulated annealing, validating its effectiveness for optimization problems such as the 3-SAT problem under the chosen parameters.

Observations and Analysis for Increasing M

The annealing solving of the 3-SAT problem for N , $M=200$ is very time-consuming. Although SA is a very strong algorithm to search through complex solution spaces and to avoid local minima, it tends to be extremely computationally intensive, since the intrinsic iterative nature is joined by heavy computational parameter tuning. With N , $M = 200$ and the above parameters, namely (`mcmc_steps` = 1000, `anneal_steps` = 85, $\beta_0 = 1.0$, $\beta_1 = 10.0$) we always converge to $c = 0.0$, so cost is always already minimized at the first steps of `anneal_steps`. Therefore, for this specific instance we may choose `anneal_steps`=15, as using a trial-and-error approach we found is more than sufficient.

As M increases, the Simulated Annealing process encounters greater challenges, reflected in higher initial costs, reduced acceptance rates, slower convergence, and frequent fluctuations to suboptimal solutions. For $M=400$ and $M=600$, the algorithm performs efficiently, converging to $c=0.0$. However, for $M=800$ and in particular, for $M=1000$, the increased density of constraints results in slower convergence and instability, with costs often reverting to non-optimal values such as $c=6.0$ and $c=7.0$. Giving an example, with `anneal_steps` = 15, and $M = 1000$ we converge to $c=9.00$, but by increasing the `anneal_steps` to 85 we can reduce it to 6.00.

This demonstrates that for higher M -values, increasing `anneal_steps` and potentially other parameters such as `mcmc_steps` or β_1 , can help mitigate the challenges posed by increased problem complexity and achieve better convergence.

3. General 3-SAT Properties

Empirical Probability of Solving 3-SAT

First of all, we should define the Empirical probability $P(N, M)$. It measures the fraction of successfully solved instances out of a fixed number of random problem instances, providing insight into the solvability of 3-SAT as the clause-to-variable ratio (M/N) changes.

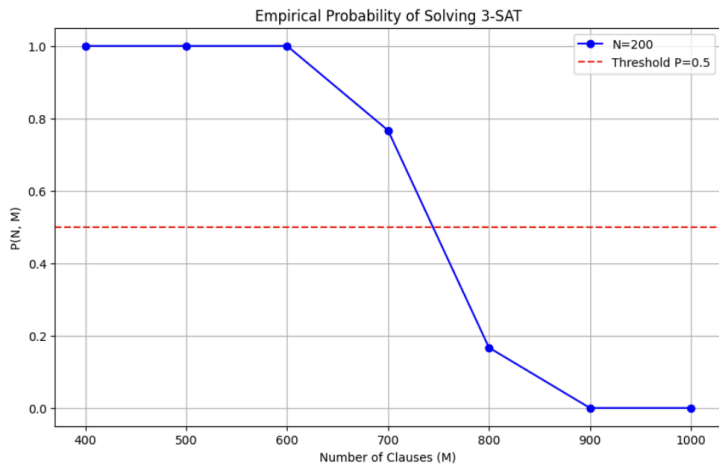


Figure 2: Empirical Probability of Solving 3-SAT with $N, M=200$

Figure 2 illustrates the Empirical probability $P(N, M)$ of solving a 3-SAT problem for $N=200$ as a function of the number of clauses $M = \{400, 500, 600, 700, 800, 900, 1000\}$

(In this case, the chosen parameters for the Simulated Annealing algorithm were: $mcmc_steps = 2500$, $anneal_steps = 20$, $\beta_0 = 0.5$, $\beta_1 = 15.0$.)

They were selected to ensure a balanced exploration of the solution space while maintaining computational efficiency

The results show that $P(N, M)$ remains close to 1 for smaller M -values, indicating a high likelihood of finding solutions when the number of clauses is relatively low. However, as M increases, a sharp decline in $P(N, M)$ is observed, with a significant drop occurring between $M=700$ and $M=800$. This region marks the transition to unsolvability, where the problem becomes increasingly constrained. The algorithmic threshold $M_{Alg}(N=200)$, defined as the M -value at which $P(N, M)=0.5$, is estimated to lie within this range $= (700, 800)$.

Beyond $M=800$, $P(N, M)$ rapidly approaches zero, reflecting the difficulty of satisfying all clauses as the clause-to-variable ratio increases. These results align with theoretical expectations and demonstrate the strong dependency of 3-SAT solvability on the number of clauses.

Moreover, a good threshold is expected to be around N times 3.5, in this case, $N=200$, so M_{Alg} should be around 700, and this is the case. Refining the M -values in the transition range (e.g. $M=725, 750, 775$) could provide an even more precise plot and therefore estimate of $M_{Alg}(N=200)$.

Empirical Probability approaching the threshold

The probability $P(N, M)$ goes from near-1 (high solvability) to near-0 (unsolvability) as the number of clauses M approaches the algorithmic threshold $MA_{lg}(N)$. Typically, at $MA_{lg}(N)$, $P(N, M)$ exhibits a sharp drop that signals the main phase transition of the 3-SAT problem. In that region, the system of variables and phrases becomes highly restricted, and the generation of configurations satisfying all restrictions is increasingly hard because of the high density of clauses. This is the point beyond which, for all practical purposes, the problem becomes unsolvable, and the probability of finding a solution becomes extremely low. The suddenness of this changeover gives an idea of the sensitivity of the problem to the clause-to-variable ratio.

Plots for different N-Values

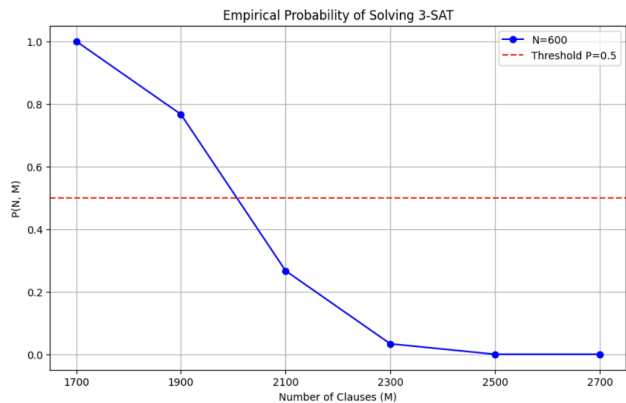
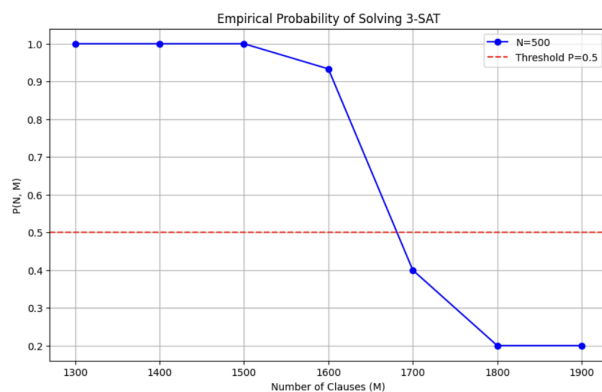
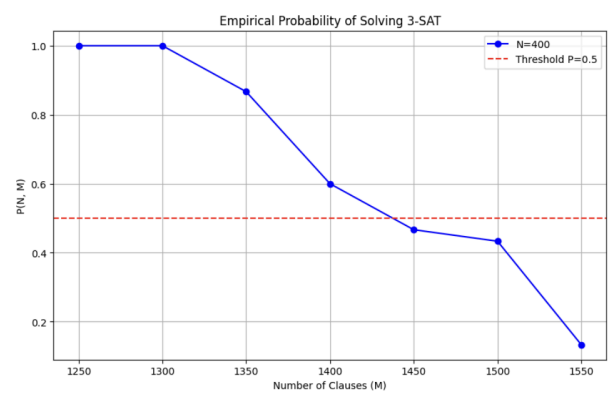
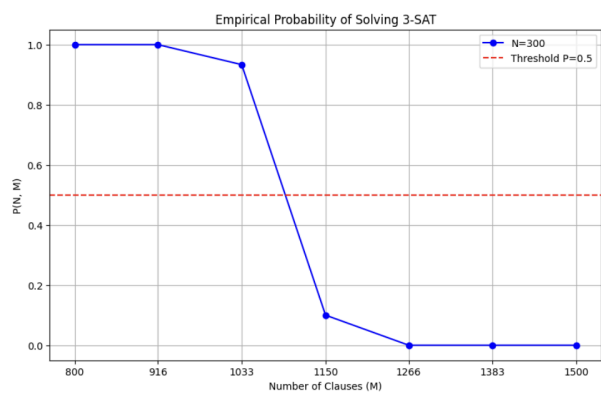


Figure 3 (Top Left): Empirical Probability of Solving 3-SAT for $N=300$

Figure 4 (Top Right): Empirical Probability of Solving 3-SAT for $N=400$

Figure 5 (Bottom Left): Empirical Probability of Solving 3-SAT for $N=500$

Figure 6 (Bottom Right): Empirical Probability of Solving 3-SAT for $N=600$

	N-Value	M-Values
Figure 3	300	800, 915, 1035, 1150, 1265, 1385, 1500
Figure 4	400	1250, 1300, 1350, 1400, 1450, 1500, 1550
Figure 5	500	1300, 1400, 1500, 1600, 1700, 1800, 1900
Figure 6	600	1700, 1900, 2100, 2300, 2500, 2700

Table 1: N-Value, M-Values

Comparison of Empirical Probability for different N-Values

(In this case, the chosen parameters for the Simulated Annealing algorithm were: `mcmc_steps` = 2500, `anneal_steps` = 20, $\beta_0 = 0.5$, $\beta_1 = 15.0$.)

The empirical probability $P(N, M)$ of solving a 3-SAT issue is constant across the investigated values of $N=300, 400, 500$, and 600 , with a clear transition from satisfiability $P(N, M) \approx 1$ to unsolvability $P(N, M \approx 0)$ as M grows. For $N = 300$, the threshold $MA_{lg}(300)$, defined as the point where $P(N, M)=0.5$, occurs around $M \approx 1050$, consistent with the scaling relationship $MA_{lg}(N) \propto 3.5$ times N . Similarly, for $N = 400$, the threshold changes to $MA_{lg}(400) \approx 1400$, whereas for $N = 500$, it is at $MA_{lg}(500) \approx 1750$. These figures show the distinctive rapid fall in $P(N, M)$ at the threshold, highlighting the transition zone when the problem becomes more limited.

The finer M -steps in these places provide a thorough perspective of the transition and indicate that it sharpens. With $N=600$, the same scaling behavior is observed, with $MA_{lg}(600) \approx 2100$. The transition from satisfiability to unsolvability remains abrupt, but compared to smaller N -values, the decline in $P(N, M)$ appears slightly broader. This may reflect the increased complexity of the solution space as N grows. Despite this, we also observed that the normalized M/N ratio at the threshold remains consistent with the observed scaling, validating the theoretical prediction of universal behavior.

Comparing all cases, the plots exhibit similar patterns of sharp transitions. The threshold is shifting proportionally to N , reinforcing the robustness of the scaling law $MA_{lg}(N) \propto 3.5$ times N and highlighting the increasing challenge of solving 3-SAT as the clause-to-variable ratio grows.

Understanding the Scaling of the Algorithmic Threshold

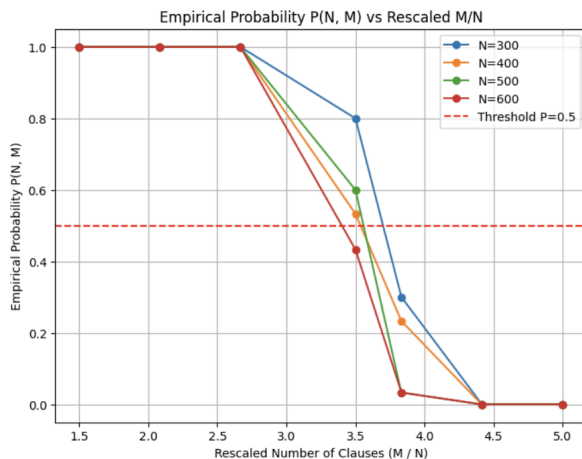


Figure 7: Empirical Probability for Rescaled M/N

For better scaling, it was decided that the M -Values change for the different N -Values.

We used `M_values=np.linspace(N×1.5, N×5,7)` inside a loop for the different N -Values.

Thus, at each iteration, we generate an array of evenly spaced numbers over the specified range.

The consistent spacing of M -values across different N -values allows the sharp drop around $MA_{lg}(N)$ to be vividly captured. The gradual overlap of rescaled curves gives evidence of the universal scaling behavior of $MA_{lg}(N)$.

Finally, we used the same Simulated Annealing parameters as the ones in the previous paragraph:

- $mcmc_steps = 2500$
- $anneal_steps = 20$
- $\beta_0 = 0.5, \beta_1 = 15.0$

This ensures that the range of clause-to-variable ratios (M/N) is broad enough to capture the transition from satisfiability $P(N, M) \approx 1$ to unsolvability $P(N, M) \approx 0$. By spanning this range, the rescaled values M/N can be compared across different N -values, allowing the identification of $MA_{lg}(N)$, the threshold where $P(N, M) = 0.5$.

	N	$M(P \approx 1)$	$M(P \approx 0.5)$	$M(P \approx 0)$
	300	450-800	1050-1150	1325-1500
	400	600-1066	1400-1533	1766-2000
	500	750-1333	1750-1916	2208-2500
	600	900-1600	2100-2300	2650-3000

Table 2: Compact Summary of Threshold ranges across different N -Values

Conclusion

Results in Table 2 are consistent with the theoretical prediction that the satisfiability threshold $MA_{lg}(N)$ scales *linearly* with N , with a constant ratio $MA_{lg}(N)/N \approx 3.5$. In this specific case, the constant ratio was ≈ 3.67 , yet it is still very close to the expected one.

This behavior reflects the universal scaling property of the satisfiability problem, which was observed earlier in the rescaled curves collapsing onto a single universal curve, and which was also predicted by the theory.

In the future, it is possible to investigate to even larger N -Values or explore the different values of K , therefore trying with the same implementation to solve K -SAT, etc., and this extension can be interesting as K fundamentally affects the complexity of the problem, e.g. it is possible to investigate on how the threshold behaves in this scenario.